Audit Trail — User Guide

Record who did what to which record, with before/after field values, browsing and export activity, and session context — without slowing down or bloating the rest of the system.

Table of Contents

1. What this module does

The module watches models you nominate and writes one **audit.log** row per recorded event. Nothing is watched by default — you declare a **watch rule** (`audit.rule`) per kind of record, per company, and confirm it.

Five actions can be recorded independently:

Action	What gets stored
`create`	One event, with one line per field that has a starting value
`write`	One event, with one line per changed field (old value + new value)
`unlink`	One event, no lines, optionally a full snapshot of the deleted record
`export`	One event listing the exported record ids, so you can reopen exactly those records
`read`	One event per record displayed, no lines

The module **only records**. It never blocks, approves, or reverses anything.

2. Installation

1. Copy `audit_trail` into your addons path.
2. **Apps → Update Apps List**.
3. Search for **Audit Trail** and click **Install**.

Depends on `base` and `web` only. No accounting, mail, or third-party module is required.

After install, an **Audit Trail** app appears in the main menu.

---

3. Roles and access

Two groups ship with the module:

Group	Can do
Auditor (`group_audit_user`)	Read `audit.log`, `audit.log.line`, `audit.session`, `audit.page.action`, and `audit.rule`. Cannot change anything, cannot open Clean-up Settings.
Audit Administrator (`group_audit_manager`)	Everything an Auditor can do, plus create/edit watch rules, open Clean-up Settings, and delete recorded history. Implies **Auditor**.

On install, every user in **Settings → Administration** (`base.group_system`) becomes an Audit Administrator, so the app is usable immediately.

Every internal user (`base.group_user`) additionally gets **read-only** access to `audit.rule`. This is deliberate: the **View Logs** button on the Contacts form and any other place a rule is referenced would otherwise raise an `AccessError` for ordinary users. Access to `audit.log` itself stays restricted to the two audit groups.

Each of `audit.rule`, `audit.log`, `audit.log.line`, `audit.session` and `audit.page.action` carries a **global company record rule**, so a user only ever sees rows belonging to a company they are allowed into.

Both groups also carry **read-only** access to Odoo's field metadata (`ir.model.fields`), which is otherwise gated behind the separate **Access Rights** permission. This is what lets the **Excluded Fields** picker on the Watch Rule form show field names for Audit Administrators editing a rule, and for Auditors reaching a rule read-only by clicking the **Rule** link on a Log event — without granting either group the ability to edit models or fields anywhere else in the database.

---

4. Setting up a watch rule

Audit Trail → Configuration → Watch Rules → New

1. **Name** — free label, e.g. `Contacts – full audit`.
2. **Watched Model** — the kind of record to watch, e.g. `Contact` (`res.partner`).
3. **Company** — one rule per model per company. A second rule for the same pair is refused with a message naming the existing rule.

4. **Detail Level** — see section 8.
    - **Full** stores the old value and the new value of every changed field.
    - **Light** stores the new value only; the **Old Value** column stays blank.
5. **Recorded Actions** — tick any of Creations, Updates, Deletions, Exports, Views.
    - **Keep Deletion Snapshot** appears once Deletions is ticked. With it on, the record's field values are stored on the event, because the record itself is gone.
6. **Exclusions**
    - **Excluded Users** — actions by these users produce **no event at all**. Typical use: an integration/API account whose traffic would drown the trail.
    - **Excluded Fields** — these fields never appear in recorded values, and are scrubbed out of deletion snapshots too. The field list is restricted to the watched model's own fields.
7. Press **Confirm**.

Nothing is recorded while the rule is in **Draft**. **Confirm** stamps **Date Confirmed** and switches the rule to **Active**. Every field except the buttons becomes read-only while the rule is active — press **Set to Draft** to edit it again.

> **Set to Draft does not delete anything.** History already recorded stays exactly where it is; the rule simply stops adding to it.

In the Watch Rules list, draft rules are shown greyed out and active rules in normal style.

---

5. Reading the audit trail

Logs

Audit Trail → Logs lists every event: **Date**, **Record**, **Technical Model**, **Action**, **User**, **Session**.

Search across record name and model in one box, and group by **User**, **Kind of Record**, **Day**, **Session**, **Action** or **Screen**. Quick filters cover Today, Last 7 Days, and each of the five actions.

Opening an event shows:

- Header facts — who, when, which record, action, detail level, watch rule.
- **Field Changes** — the changed fields side by side (Field / Old Value / New Value). For a creation there is no old value; for a Light-detail update the old value is blank by design.
- **Deletion Snapshot** — for deletions recorded with the snapshot option, the stored field values as JSON.
- **Exported Records** — for export events, plus a **View Exported Records** button that opens exactly the records that were in that export.

Field changes across every model at once

Audit Trail → Field Changes lists `audit.log.line` rows directly, with the parent event's date, record and user for context. This is the screen to use when the question is "who changed a phone number last week" rather than "what happened to this record" — you can search on old and new values across every watched model at the same time.

From the record itself

On a Contact form, Auditors see a **View Logs** button in the button box. It opens the Logs list filtered to that exact record, grouped by action.

---

6. Sessions and screens

A **session** (`audit.session`) groups everything one user did against watched records until they stopped for longer than the inactivity timeout (30 minutes by default). Activity inside the window extends the open session; a longer gap starts a new one.

To change the timeout, set the system parameter `audit_trail.inactivity_minutes` (**Settings → Technical → System Parameters**).

Audit Trail → Sessions lists sessions grouped by user. Opening one shows:

- **Screens Visited** — the `audit.page.action` trail within that session.
- **Events** — the session's full story in order.

---

7. Retention clean-up

Audit Trail → Configuration → Clean-up Settings

Clean-up is **off by default** — nothing is ever deleted automatically until you turn it on.

Setting	Default	Meaning
Enable Automatic Clean-up	Off	Master switch
Keep Events For (days)	180	Events older than this are deleted
Batch Size	1000	Events deleted per run

Pressing **Save** writes the settings and enables or disables the scheduled job **Audit Trail: Retention Clean-up** to match. **Save & Run Now** additionally performs one clean-up immediately, after a confirmation prompt.

Each run works through three passes, in order, all bounded by the same **Batch Size**:

1. **Events** — `audit.log` rows older than the retention period (deleting an event cascades to its field-change lines).
2. **Sessions** — `audit.session` rows whose last activity is older than the retention period **and** that

no longer have any event pointing at them. A session with events left is never removed, even if it is old — its own events keep it alive until they expire in pass 1 on a later run.

3. **Screens** — `audit.page.action` rows whose own **last activity** is older than the retention period and that no longer have any event pointing at them. A screen is aged by the logical timestamp of the last event recorded on it, not by when its row happened to be created, so seeded, imported or migrated screens expire together with the events they belong to instead of being shielded indefinitely. This also catches a screen whose events have all expired while its session is still active because of other, newer screens — a case session clean-up alone would miss.

When any of the three passes fills its batch, the job re-triggers itself for the next batch. A first clean-up on a large history therefore proceeds in short transactions instead of one long lock. Without this, **Sessions** and **Screens** would keep growing indefinitely no matter how `retention_days` is set, since nothing previously retired them.

Worked example. Retention 180 days, batch size 1000, and 4,300 events older than the cut-off: the job deletes 1000, re-triggers, deletes 1000, and so on — five runs in total, the last removing 300 events and stopping because the batch was not full. Once events stop filling the batch, later runs start clearing out whatever sessions and screens have been left empty by that history.

---

8. Performance: what each option costs

The module is built so that a model **without** an active rule costs essentially nothing: the only work done is a cached set-membership check on the model name.

Two options are genuinely expensive, and both are flagged on screen next to the control that turns them on:

Full detail re-reads each record before every write to that model, in order to capture the old value. On a model that is written to constantly, prefer **Light**, which skips that read and stores new values only.

Recording views creates one event per row displayed. Opening a list of 80 contacts records 80 events. This is off by default and should stay off unless you specifically need read auditing.

Other deliberate limits:

- Recorded values are truncated at 2,000 characters.
- Binary fields and one-to-many fields are never recorded (they are large and derived respectively).
- Excluded users are filtered out at the point of recording, so an excluded account costs nothing beyond the rule lookup.

---

9. Demo data

Installing with demo data gives you a working example immediately:

- An active watch rule **Contacts - full audit** on `res.partner`, full detail, with Creations, Updates, Deletions and Exports ticked, Views off, deletion snapshots on, and an **Integration User** on the exclusion list.
- Five demo contacts (Northwind Traders, Lakeside Consulting, Bridgeport Analytics, Orchard Supply Co., Marek Halloway).
- 17 recorded events: 4 creations, 9 updates with realistic old/new values, one deletion carrying a snapshot, one export covering 4 contacts, and 2 view events.
- Three sessions across two users, each linked to its screens and its events.

---

10. Limitations

Stated plainly, because knowing these up front is worth more than a longer feature list:

- **No tamper-proofing.** Audit Administrators can delete recorded history. This is an activity record, not a sealed vault.
- **No approval or blocking.** The module records; it never prevents an action.
- **No alerting.** Nothing is emailed or notified on any event.
- **No dashboards or charts.** List, group and search only.
- **No per-person allow-list.** You can exclude users, not restrict watching to a named set of users.
- **Events share the transaction that produced them.** If the user's operation is rolled back, its audit event is rolled back with it. Conversely, a failure inside the audit code can never abort the user's operation — it is caught and logged to the server log instead.
- **Screen attribution is per model, per session.** Two different Odoo actions on the same model within one session are recorded against a single `audit.page.action` row.
- **Read tracking is skipped on read-only database cursors.** Deployments that route read requests to a replica will not record view events from those requests.
- **Watch rules cannot be deleted through the UI** — Audit Administrators can create and edit them, and set them back to Draft to stop recording.
- **Saving a record from the form view** produces an update event; the form's subsequent re-read is deliberately suppressed so it does not also produce a view event.

Generated by OdoMate — https://odomate.pro